Senior Backend Engineer — Node.js/TypeScript

Stage 2 — Practical Build Task

Sent to Stage 1 qualified candidates only | MeridianSquare.Global

This stage is sent only to candidates who cleared Stage 1. Well done on the written questions — we are now looking to see you execute.

Instructions

- Build the working code described below using the required stack
- You may use any AI tools — submit key prompts and a 300-500 word AI usage review
- Cap effort at 3-4 hours. Submit within 5 days
- If you run out of time, submit what you have with a clear note on what remains
- GitHub repo may be public or private. If private, add [INSERT GITHUB HANDLE] as collaborator

Code for evaluation only: This code is for evaluation purposes only and will not be used in our product.

Platform Context

CubeSquare's investor KYC flow integrates with an external identity provider. When an investor completes verification, the provider sends a webhook to our platform. This webhook must be handled idempotently, securely, and without blocking the response. The investor's KYC status in our database must then be updated, and downstream workflows triggered asynchronously.

Build Task — KYC Webhook Integration Service

Stack (required): Node.js + TypeScript Express or Fastify PostgreSQL + Prisma (starter schema provided below) Redis (for idempotency and queue)

Prisma Starter Schema

Use and extend this schema. Do not redesign it — the database engineer owns the schema. Add only what the webhook handler requires.

```
model Tenant {
```

```
  id         String     @id @default(uuid())
  name       String
  investors  Investor[]
}

model Investor {
  id          String   @id @default(uuid())
  tenant_id   String
  email       String   @unique
  kyc_status  String   @default("pending")
  tenant      Tenant   @relation(fields: [tenant_id], references: [id])
}

model WebhookEvent {
  id           String   @id @default(uuid())
  event_id     String   @unique
  provider     String
  payload      Json
  processed_at DateTime @default(now())
}
```

Endpoint 1 — POST /webhooks/kyc

The KYC provider sends this payload:

```
{
  "event_id": "evt-uuid-abc123",
  "investor_id": "inv-uuid-456",
  "new_status": "approved",
  "timestamp": "2026-06-20T14:30:00Z"
}
```

The provider signs each webhook with HMAC SHA-256. The signature is sent in the X-KYC-Signature header. The signing secret is KYC_WEBHOOK_SECRET (environment variable).

The handler must:

- Validate the HMAC signature — reject with 401 if invalid, log the attempt
- Check Redis for the event_id — if already processed, return 200 OK immediately without reprocessing
- Return 200 OK immediately after validation — do not wait for the database update
- Enqueue the status update asynchronously (use BullMQ, a Redis-backed queue, or equivalent)
- The async worker must: update investor kyc_status in PostgreSQL, store the event in WebhookEvent for audit, clear the Redis idempotency key after successful processing

Response Requirements

Code	Condition	Expected behaviour

200	Webhook already processed	Return 200 OK immediately — do not reprocess
200	Webhook processed successfully	Return 200 OK after enqueueing the status update
400	Invalid payload	Missing required fields or invalid format
401	Invalid signature	HMAC signature does not match — reject and log
500	Internal error	Unexpected failure — return 500, do not acknowledge

Endpoint 2 — GET /investors/:id/kyc-status

Returns the KYC status of an investor. Requirements:

- Requires Bearer token authentication — JWT containing tenant_id and investor_id claims
- The authenticated tenant must match the investor's tenant_id — 403 if not
- Returns: investor id, email, kyc_status, and the timestamp of the most recent WebhookEvent for this investor if one exists
- Correct status codes: 200, 401, 403, 404, 500

Additional Requirements

- TypeScript throughout — no any
- Typed interfaces for the webhook payload, JWT claims, and API responses
- Environment variables: DATABASE_URL, REDIS_URL, KYC_WEBHOOK_SECRET, JWT_SECRET
- README with setup and run instructions — reviewer must be able to run with two commands

Deliverables

1. GitHub repository with working code and README
2. Architecture note (200-350 words): decisions you made, tradeoffs accepted given the timebox, what you would add next
3. AI usage review (300-500 words): tools used, key prompts, what AI got right, what you corrected and why

Evaluation Weighting — Stage 2

Category	Weight
Webhook idempotency implementation — Redis pattern, async processing, edge cases	**30%**

Tenant-isolation enforcement on GET endpoint — middleware pattern, error handling	**25%**
TypeScript discipline and code quality	**20%**
Architecture note — reasoning and tradeoff clarity	**15%**
AI usage quality and critical review	**10%**

Timebox: 3-4 hours. Submit within 5 days. An incomplete but well-structured submission outscores a complete but shallow one.